

Overview

This Python program is designed to recognize handwritten digits using a neural network model built with **TensorFlow/Keras** and tested on custom digit images saved in a folder. It makes predictions on input images and visualizes the results.

Dependencies

The code uses the following libraries:

- **os** → for handling file paths and iterating over images.
- **cv2 (OpenCV)** → for image processing (loading PNG files).
- **numpy** → for array manipulations.
- **matplotlib** → for visualizing the processed images.
- **tensorflow** → for building, training, saving, and loading the deep learning model.

Code Explanation

1. Importing Required Libraries

```
import os  
  
import cv2  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import tensorflow as tf
```

These libraries are used for data loading, preprocessing, visualization, and model operations.

2. (Commented Out) Model Training Block

```
# mnist = tf.keras.datasets.mnist  
  
# (x_train, y_train), (x_test, y_test) = mnist.load_data()  
  
  
# x_train = tf.keras.utils.normalize(x_train, axis=1)  
# x_test = tf.keras.utils.normalize(x_test, axis=1)  
  
  
# model = tf.keras.models.Sequential()  
# model.add(tf.keras.layers.Flatten(input_shape = (28, 28)))  
# model.add(tf.keras.layers.Dense(128, activation='relu'))
```

```
# model.add(tf.keras.layers.Dense(128, activation='relu'))

# model.add(tf.keras.layers.Dense(10, activation='softmax'))

# model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

# model.fit(x_train, y_train, epochs=3)

# model.save('handwritten.model')
```

- Loads the **MNIST dataset** (handwritten digits: 0–9).
- Normalizes pixel values (0–255 → 0–1).
- Builds a **Sequential Neural Network**:
 - Flatten layer (converts 28×28 image into a 784-element vector).
 - Two dense (fully connected) hidden layers with **ReLU activation**.
 - Output layer with **Softmax activation** (10 classes → digits 0–9).
- Compiles the model with **Adam optimizer** and **sparse categorical cross-entropy loss**.
- Trains for **3 epochs**.
- Saves the model as **handwritten.model**.

(This block is commented out, meaning the program assumes the model is already trained and saved.)

3. Loading the Trained Model

```
model = tf.keras.models.load_model('handwritten.model')
```

- Loads the pre-trained model for inference (digit prediction).

4. Prediction on Custom Images

```
image_number = 1

while os.path.isfile(f"digits/digit{image_number}.png"):

    try:

        img = cv2.imread(f"digits/digit{image_number}.png")[:, :, 0]

        img = np.invert(np.array([img]))
```

```

prediction = model.predict(img)
print(f"This digit is probably a {np.argmax(prediction)}")
plt.imshow(img[0], cmap = plt.cm.binary)
plt.show()
except:
    print("Error!")
finally:
    image_number += 1

```

Step-by-step:

1. Iterate through images

- Starts with digit1.png and continues as long as the file exists in the digits/ folder.

2. Read and preprocess image

- Loads image using OpenCV (cv2.imread).
- Extracts the first channel ([:, :, 0]) → converts it to grayscale.
- Inverts the colors (MNIST digits are white on black, but your images might be black on white).
- Wraps in a numpy array to fit model input requirements: (1, 28, 28).

3. Prediction

- Calls model.predict(img) to get probabilities for digits 0–9.
- Uses np.argmax(prediction) to get the most likely digit.

4. Visualization

- Displays the processed image with Matplotlib.
- Prints predicted digit in console.

5. Error Handling

- If something fails (wrong file, format, etc.), prints "Error!" but still continues.

Program Flow

1. Load trained digit recognition model.
2. Check for digit images in digits/ folder.

3. For each image:
 - Read and preprocess.
 - Predict digit using neural network.
 - Print result and display image.
4. Repeat until no more images are found.

Possible Improvements

- Fix typos in training code:
 - `tf.keras.utis.normalize` → should be `tf.keras.utils.normalize`
 - `tf.keras.models.sequential()` → should be `tf.keras.models.Sequential()`
 - `sparse_categorical_crossentropy"` → should be `'sparse_categorical_crossentropy'`
- Resize external images to **28×28 pixels** before prediction.
- Add **confidence scores** (e.g., `np.max(prediction)`).
- Use **try-except for preprocessing** more specifically (e.g., catching shape errors).
- Create a **GUI or Flask API** for real-time digit prediction.